Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 12.11.2024.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2024_Test1_GrupaA_IT1g2023**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Quick Sort*

Napisati ***quickSort*** metodu ***Quick Sort*** algoritma za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalazi klasa ***RandomArrayGenerator*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- U okviru klase ***QuickSort*** napisati implementaciju za metodu ***private static void quickSort(int[] arr, int lower, int upper)***
- Obezbediti kod u **Main** metodi za testiranje algoritma.

Zadatak 2 – *Maksimalan zbir blizanaca*

U povezanoj listi sa **parnim brojem čvorova** veličine n, definišemo **blizanački par** čvorova na sledeći način:

Za **i-ti čvor** (počevši od indeksa 0) postoji odgovarajući **blizanac** na poziciji (n−1−i), ako važi 0 <= i <= (n/2)-1.

Na primer:

- Kada je n=4, čvor sa indeksom 0 ima blizanca u čvoru sa indeksom 3, dok čvor sa indeksom 1 ima blizanca u čvoru sa indeksom 2. Ova dva para su jedini blizanački parovi za listu veličine 4.

Zbir blizanaca definiše se kao zbir vrednosti čvora i njegovog blizanca. Zadatak je da pronađete **najveći zbir blizanačkih parova** u listi. Rešenje treba da pronađe maksimalni zbir blizanaca u O(n) vremenu.

Dodatni zahtevi za rešavanje zadatka:

- Algoritam pisati u okviru klase ***LinkedList.***
- Zahtevan potpis metode: ***public int findMaxTwinSum()***

Primer 1: Input: 5 ↔ 4 ↔ 2 ↔ 1
Output: 6

Primer 2: Input: 4 ↔ 2 ↔ 2 ↔ 3
Output: 7

Zadatak 3 – *Provera dva reda*

Za dva reda *queue1* i *queue2* treba proveriti da li možemo dobiti *queue2* iz *queue1* koristeći samo operacije definisane u okviru klase ***Queue***. Elementi koji se nalaze u drugom redu moraju pratiti isti redosled kao u prvom redu, ali neki elementi iz prvog reda mogu biti preskočeni.

Dodatni zahtevi za rešavanje zadatka:

- Zadatak rešavati pomoću strukture reda (klasa *Queue*) koja je data u okviru šablona zadatka.
- Zahtevan potpis metode: ***public bool CanFormQueue(Queue queue2)***

<u>Primer:</u>

- Red1: [1, 2, 3, 4, 5]
- Red2: [2, 3, 4]
- Ispis: true

- Ulazni niz: [1, 2, 3, 4, 5]
- Izlazni niz: [1, 4, 2]
- Ispis: false